OPERATING SYSTEMS
PROJECT REPORT

PAGE REPLACEMENT ALGORITHM SIMULATOR

Implementation and comparative analysis of FIFO, LRU, and Optimal page replacement

Submitted by
Siddh Jain
Roll No.: BTE25DTA000039
B.Tech — Data Analytics
SAGE University
Operating Systems Lab

A practical study of virtual-memory page replacement policies, simulation steps, and performance metrics.

1. Abstract

This project simulates three page replacement strategies used in the study of virtual memory: First-In, First-Out (FIFO), Least Recently Used (LRU), and Optimal Page Replacement. Given a page-reference sequence and a number of memory frames, the simulator records the frame state after each reference and reports page faults, page hits, hit ratio, and fault ratio. The experiments help illustrate the differences between the algorithms and demonstrate Belady's anomaly for FIFO.

2. Aim and Objectives

Aim: To implement and compare FIFO, LRU, and Optimal page replacement algorithms using Python and a Jupyter Notebook.

• Understand page faults and page hits in demand paging.

• Track frame contents as each page reference is processed.

• Calculate and compare page-fault counts and performance ratios.

• Observe Belady's anomaly using a reference sequence with different frame counts.

• Practice documenting experiments and interpreting results.

3. Background

Demand paging loads a page into main memory when it is needed. If a requested page is not currently in a memory frame, a page fault occurs. If the page is already present, the reference is a page hit. When a fault occurs and all frames are occupied, the replacement policy selects a resident page to remove before loading the requested page.

4. Algorithms

4.1 FIFO — First-In, First-Out

FIFO replaces the page that has been in memory the longest, based on arrival order. It is straightforward to implement with a queue or circular pointer. Its main limitation is that it may remove a page that will soon be needed again, and it can exhibit Belady's anomaly.

4.2 LRU — Least Recently Used

LRU replaces the page whose most recent access occurred farthest in the past. It uses past access history as a guide to future needs and requires additional recency bookkeeping, such as timestamps, counters, or an ordered data structure.

4.3 Optimal Page Replacement

Optimal replaces the page whose next use is farthest in the future, or a page that will not be used again. For a known reference sequence, it gives the minimum possible page-fault count and serves as a benchmark. It cannot be implemented exactly as an online policy in a real operating system because future references are not known in advance.

5. Implementation Approach

The program accepts a page-reference string and a positive frame count. It validates the input and runs each algorithm on the same sequence so their results can be compared fairly. For each reference, the simulation records the step number, requested page, frame contents, hit/fault status, and replaced page. After processing the sequence, it calculates total faults, total hits, hit ratio, and fault ratio.

Hit ratio = Page hits ÷ Total page references

Fault ratio = Page faults ÷ Total page references

The hit ratio and fault ratio sum to 1 (or 100%) when all references are classified as either hits or faults.

6. Experiment 1 — Manual Trace Results

Reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2
Number of frames: 3

Algorithm	Page faults	Page hits	Hit ratio	Fault ratio
FIFO	10	3	23.08%	76.92%
LRU	9	4	30.77%	69.23%
Optimal	7	6	46.15%	53.85%

For this reference string, Optimal has the fewest page faults, followed by LRU and then FIFO. This is consistent with Optimal's role as a benchmark when the future reference sequence is known.

7. Experiment 2 — Belady's Anomaly

Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Algorithm	Faults with 3 frames	Faults with 4 frames
FIFO	9	10
LRU	10	8
Optimal	7	6

FIFO exhibits Belady's anomaly for this input: its page-fault count rises from 9 to 10 when the frame count increases from 3 to 4. LRU and Optimal do not exhibit this anomaly because they have the stack property: increasing the number of frames does not increase their page-fault count for the same reference string.

8. Experiment 3 — Custom Reference Strings

Case	Reference string	Frames	FIFO faults	LRU faults
LRU better	5 4 1 5 1 2 5 1 5 5 4 1 2 1 5 2 3 4	3	11	9
FIFO equal	5 1 5 3 5 2 1 5 5 2 3 1 5 1 5 1 5 2	4	4	4

9. Short Questions

Q1. Why can Optimal not be implemented exactly in a real operating system?

It requires exact knowledge of future page references to determine which page will be used farthest in the future. A real operating system cannot know that sequence in advance, so Optimal is mainly used as a theoretical benchmark.

Q2. What extra bookkeeping does LRU need compared with FIFO?

LRU tracks recent use through timestamps, counters, or an ordered stack/list. FIFO mainly tracks arrival order, so LRU generally requires more bookkeeping.

Q3. If the number of frames equals the number of distinct pages, how many faults occur?

Assuming frames start empty, there is one fault for each distinct page on its first reference. Once all distinct pages are loaded, subsequent references hit because all of them fit in memory.

10. Conclusion

The simulator demonstrates that page replacement policies can produce different fault counts for the same reference sequence. FIFO is simple but may exhibit Belady's anomaly. LRU uses recent access history to guide replacement, while Optimal provides a theoretical lower-bound benchmark when future references are known. The experiments reinforce the importance of page replacement policy in virtual-memory management and provide practical experience with algorithm tracing and performance measurement.

11. Suggested Validation Before Submission

• Run the notebook from the first cell to the last in your own environment.

• Verify the displayed output against the result tables in this report.

• Add genuine screenshots of your simulator running, if required by the assignment.

• Ensure the repository contains the files referenced in the README and report.